some constant ratio of output to input intervals for all points, whereas nonlinear transformations will have a varying ratio.

Linear transformations are often represented as *matrices*, which are rectangular grids of numbers. Matrices encode the coefficients for multivariable linear functions, such as

$$f^x(x,y) = Ax + By \; f^y(x,y) = Cx + Dy$$

While this appears to be two functions, this is really a single function that maps a 2-dimensional point (x,y) to a new 2-dimensional point (x',y') using the coefficients A,B,C,D. To find x, you use the f^x function, and to find y' you use the f^y function. We could have written this as a single line:

$$f(x,y) = (Ax + By, Cx + Dy)$$

This makes it more clear that the output is a 2-tuple or 2-dimensional vector. In any case, it is useful to think of this function in two separate pieces since the computations for the x and y components are independent.

While the mathematical notion of a vector is very general and abstract, for machine learning a vector is just a 1-dimensional array of numbers. This linear transformation takes a 2-vector (one that has 2 elements) and turns it into another 2-vector, and to do this it requires four separate pieces of data, the four coefficients. There is a difference between a linear transformation like $Ax + By$ and something like $Ax + By + C$ which adds a constant; the latter is called an *affine* transformation. In practice, we use affine transformations in machine learning, but for this discussion we will stick with just linear transformations.

Matrices are a convenient way to store these coefficients. We can package the data into a 2 by 2 matrix:

$$F = \begin{bmatrix} A & B \\ C & D \end{bmatrix}$$